

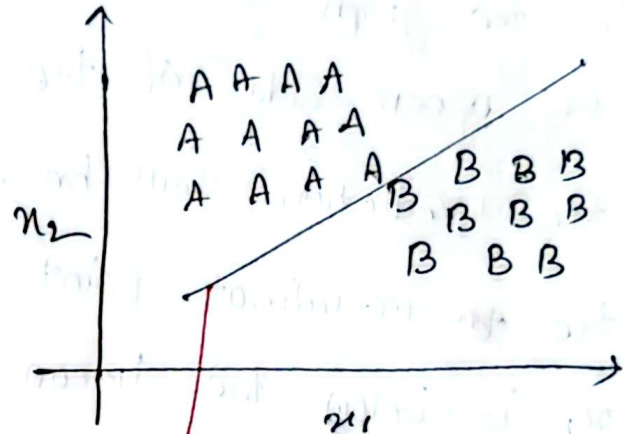
Logistic Regression

Unlike linear regression, Logistic regression is used to predict categorical outcomes. So, Logistic Regression is use for classification problems.

Let's say we have a dataset with two features x_1 and x_2 and with class A, B $\rightarrow$

x_1	x_2	class
-	-	A
-	-	B
-	-	A
...	...	A

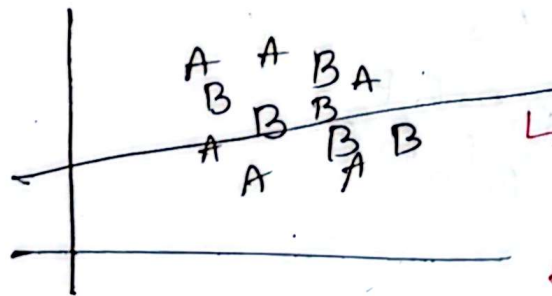
Plot $\rightarrow$



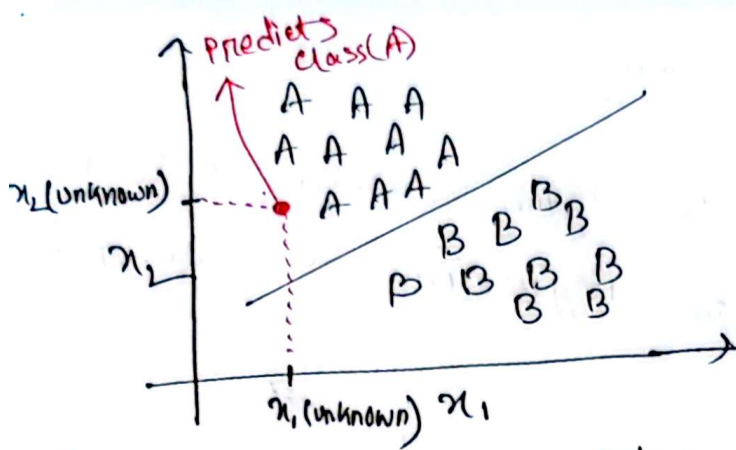
This data is linearly separable by this line.

We can apply logistic regression, if the data is linearly separable by a line (in two dimension) or hyperplane (in higher dimension)

If data is non linear like $\rightarrow$



$\rightarrow$ we can't linearly separate the class A, B with this line.

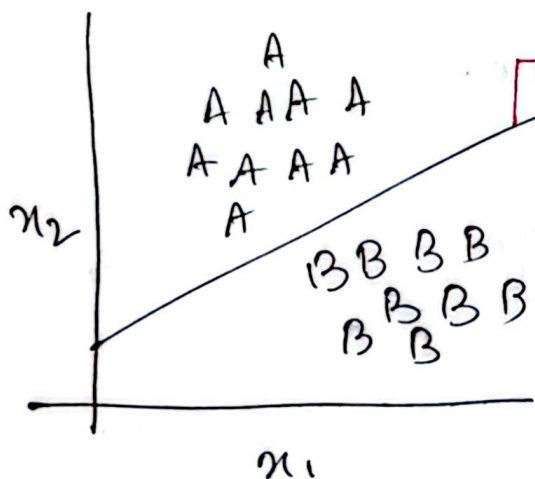


To predict for a new value of x_1 and x_2 we can ~~just~~ easily use

this model. For two values $x_1(\text{unknown})$ and $x_2(\text{unknown})$ in the graph we can see we get the coordinate at the upper side of the ~~sep~~ linear line (our model). So the prediction will be class A.

If the ~~the~~ coordinate point for a new unknown x_1, x_2 is below the linear line ~~A~~ will be the prediction will be class B.

Here this linear line is our logistic regression model, which we need to find. Basically, we need to find the equation of this linear line.



The equation will be,

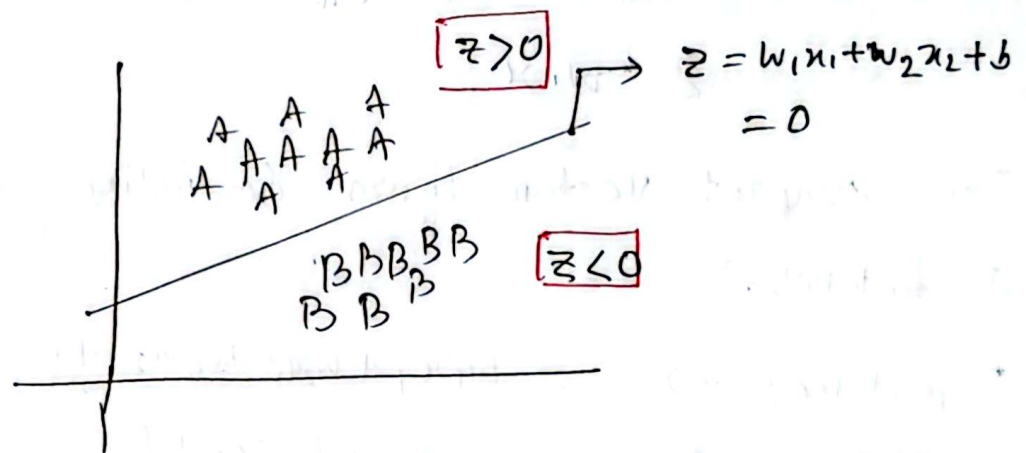
$$z = x_1 w_1 + x_2 w_2 + b$$

for two features x_1 and x_2

Just like linear regression we need to find the weights here $\rightarrow w_1, w_2$ and b .

For higher dimension like for 3 features, the equation will be $z = x_1 w_1 + x_2 w_2 + x_3 w_3 + b$.

Now ~~for~~^{over} this line / equation $z = w_1 x_1 + w_2 x_2 + b$, we will get $z = 0$.



So, the upper region of the line is $z > 0$ and lower region of the line is $z < 0$.

So, if $z > 0$, the predicted class will be A

if $z < 0$, the predicted class will be B.

So, using this equation we can easily find prediction for any new ~~value~~ unknown value of x_1 and x_2 .

if $w_1 x_1(\text{unknown}) + w_2 x_2(\text{unknown}) + b > 0 \rightarrow \text{Class A}$

if $w_1 x_1(\text{unknown}) + w_2 x_2(\text{unknown}) + b < 0 \rightarrow \text{Class B}$.

We can use vector notations for this linear equation.

The input vector $\rightarrow \langle x_1, x_2 \rangle = x$

and weight vector $\rightarrow \langle w_1, w_2 \rangle = w$

So, we can write this $w_1 x_1 + w_2 x_2$ part as vector dot product $\rightarrow$

$$w_1 x_1 + w_2 x_2 = w \cdot x$$

So, we can rewrite the equation as,

$$z = w \cdot x$$

This compact vector form generalizes the number of features $\rightarrow$

$$3 \text{ features} \rightarrow z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

$$2 \text{ features} \rightarrow z = w_1 x_1 + w_2 x_2 + b$$

$$1 \text{ feature} \rightarrow z = w_1 x_1 + b$$

$$\therefore \text{for } n \text{ feature } z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$
$$= \left(\sum_{i=1}^n w_i x_i \right) + b$$

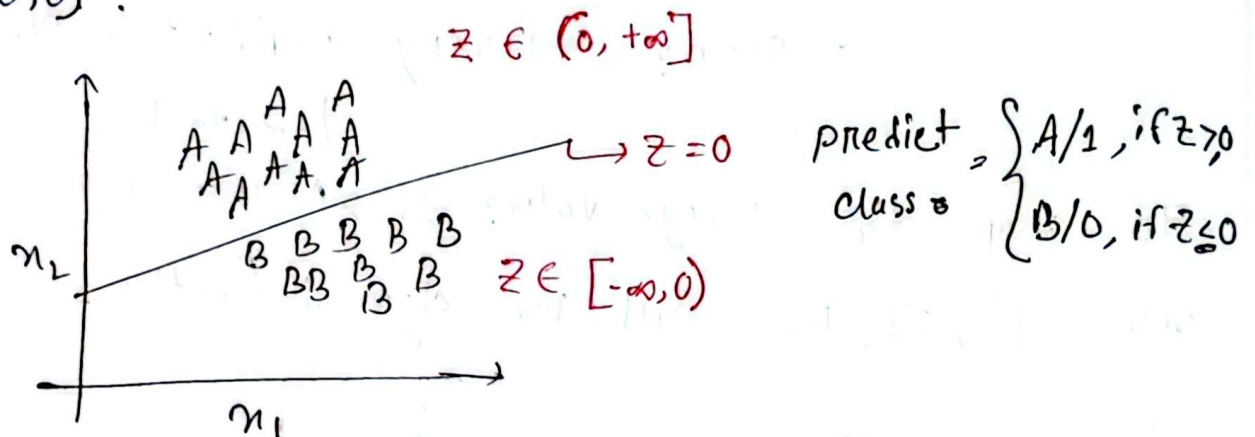
$$z = w \cdot x + b$$

$$\text{where } w = \langle w_1, w_2, w_3, \dots, w_n \rangle$$

$$x = \langle x_1, x_2, \dots, x_n \rangle$$

Why do we need sigmoid function?

Now this linear equation z can take any value from $[-\infty, +\infty]$. We have already seen $z=0$ over the line. Above the line $z > 0$, and the range is $[0, +\infty]$. Below the line z can take $[-\infty, 0]$.



But instead of taking large value we want to take the probabilistic values. For example, $P(y=A | x_1, x_2)$
 $= P(y=A | x) =$ Probability of class A for a given value of feature x_1 and x_2 .

It is because logistic regression is probabilistic not just binary. The probability gives ~~us~~ ^{the} confidence of a prediction to being in a class.

$P(y=A | x) = 0.993 \rightarrow$ very confident in class A

$P(y=A | x) = 0.55 \rightarrow$ slightly confident in class A

$P(y=A|x) = 0.007 \rightarrow$ ^{less} very confident in class A,
on very confident in class B.

in contrast,

$$P(y=B) = 1 - P(y=A)$$

$$= 1 - 0.007$$

$$= 0.993 \rightarrow \text{very confident in class B.}$$

$[-\infty, +\infty]$

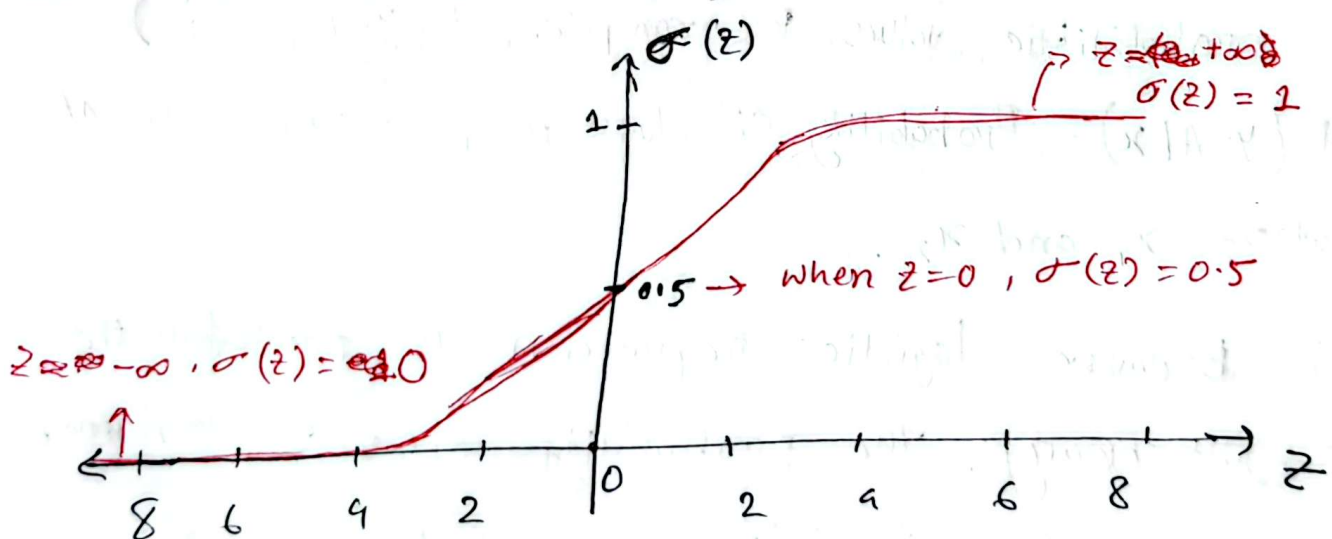
To map the large values of z into probabilistic value $[0, 1]$, we will use sigmoid function.

$$\text{Sigmoid Function } \sigma(z) = \frac{1}{1 + e^{-z}}$$

Where,

$$z = W \cdot x + b.$$

Let us plot $\sigma(z)$ vs $z \rightarrow$



so, sigmoid function ~~re~~ brings the $[-\infty, +\infty]$ values into the range of $[0, 1]$

Till now we have predicted class y ,

$$\text{Prediction } \hat{y} = \begin{cases} A, & \text{if } z > 0 \rightarrow +\infty \\ B, & \text{if } z < 0 \rightarrow -\infty \end{cases}$$

No, we will use the sigmoid function $\rightarrow$

$$\text{Prediction } \hat{y} = \begin{cases} A, & \text{if } \sigma(z) > 0.5 \rightarrow +1 \\ B, & \text{if } \sigma(z) < 0.5 \rightarrow +0 \end{cases}$$

$$\text{here } z = w \cdot x + b$$

$\downarrow$
the previous linear line
use as separator.

So, we can calculate, the probability of class A,

$P(y=A|x)$ with the sigmoid function.

$$\therefore \hat{y} = P(y=A|x) = \sigma(z) = \sigma(w \cdot x + b)$$

previously we have seen, probability of class B,

$$P(y=B|x) = 1 - P(y=A|x)$$

$$= 1 - \sigma(z)$$

$$= 1 - \sigma(w \cdot x + b)$$

Let us use class 1 and class 0, instead of class A and class B.

$$\therefore \boxed{P(y=1|x) = \sigma(z) = \sigma(w \cdot x + b)}$$

$$= \frac{1}{1 + e^{-(w \cdot x + b)}}$$

$$\text{and } \boxed{P(y=0|x) = 1 - \sigma(z)}$$

$$= 1 - \sigma(w \cdot x + b)$$

$$= 1 - \frac{1}{1 + e^{-(w \cdot x + b)}}$$

$$= \frac{1 + e^{-(w \cdot x + b)} - 1}{1 + e^{-(w \cdot x + b)}}$$

$$= \frac{e^{-(w \cdot x + b)}}{1 + e^{-(w \cdot x + b)}}$$

$$\Rightarrow 1 - \sigma(z) = \boxed{\frac{e^{-z}}{1 + e^{-z}}} \quad \text{--- (1)}$$

$$= \frac{e^{-z} \times e^z}{(1 + e^{-z}) \times e^z}$$

[multiplying nominator and denominator by e^z]

$$= \frac{1}{e^z + 1} \Rightarrow \left[\cancel{e^{-z}} \times \frac{1}{\cancel{e^{-z}}} \right]$$

$$= \frac{\frac{1}{e^z} \times e^z}{e^z + \frac{1}{e^z} \times e^z}$$

$$\left[e^{-z} = \frac{1}{e^z} \right]$$

$$= \frac{1}{e^z + 1}$$

$$= \frac{1}{1+e^z}$$

$$= \frac{1}{1+e^{-(-z)}} \quad [-(-z)=+z]$$

$$= \sigma(-z) \quad \left[\sigma(z) = \frac{1}{1+e^{-z}} \right]$$

$$\therefore \boxed{1 - \sigma(z) = \sigma(-z)}$$

Derivative of $\sigma(z)$:

$$\frac{d}{dz} \sigma(z) = \frac{d}{dz} \left(\frac{1}{1+e^{-z}} \right)$$

$$= \frac{d}{dz} (1+e^{-z})^{-1}$$

$$= -1 \times (1+e^{-z})^{-2} \times \frac{d}{dz} (1+e^{-z}) \quad \text{[Chain rule]}$$

$$= -(1+e^{-z})^{-2} \times (-e^{-z})$$

$$= e^{-z} \times (1+e^{-z})^{-2}$$

$$= \frac{e^{-z}}{(1+e^{-z})^2}$$

$$= \frac{1}{(1+e^{-z})} \times \frac{e^{-z}}{(1+e^{-z})}$$

$$= \sigma(z) \times (1 - \sigma(z)) \rightarrow \text{From (1)}$$

$\therefore$ derivative of sigmoid function

$$\boxed{\frac{d}{dz} \sigma(z) = \sigma(z) (1 - \sigma(z))}$$

————— (4)

Example:

* A sample text document showing extracted features in the vector x →

It's hokey. There are virtually no surprises, and the writing is second-rate. So why was it so enjoyable? For one thing, the cast is great. Another nice touch is it is the music. I was overcome with the urge to get off the couch and start dancing. It sucked me in, and it'll do the same to you.

Annotations:

- $x_1 = \text{negative lexicon word count} = 2$ (points to "hokey", "second-rate")
- $x_2 = \text{positive lexicon word count} = 1$ (points to "enjoyable")
- $x_3 = \text{if there is 'no'} = 1$ (points to "no")
- $x_4 = \text{First and second pronoun count} = 3$ (points to "I", "me", "you")
- $x_5 = \text{if there is 'I'} = 0$
- $x_6 = \ln(\text{word count}) = \ln(66) = 4.19$

Find out if this paragraph/text carries positive emotion or negative emotion.

The weight vector,

$$W = \langle w_1, w_2, w_3, w_4, w_5, w_6 \rangle$$

$$= \langle 2.5, -5.0, -1.2, 0.5, 2.0, 0.7 \rangle$$

and bias $b = 0.1$.

Solution:

<u>Feature Set</u>	<u>Definition</u>	<u>Value</u>	<u>Value</u>
x_1	count(positive lexicon words $\in$ doc)	3	
x_2	count(negative lexicon words $\in$ doc)	2	
x_3	$\begin{cases} x_3 = 1 & \text{if "no" } \in \text{ doc} \\ x_3 = 0 & \text{if otherwise} \end{cases}$	1	↓ there is 'No' in the doc
x_4	Count (1st and 2nd pronoun)	3	
x_5	$\begin{cases} x_5 = 1 & \text{if "!" } \in \text{ meta Doc} \\ x_5 = 0 & \text{if otherwise} \end{cases}$	0	↓ there is no sign
x_6	$\ln(\text{word count of doc})$	$\ln(66) = 4.19$	↓ there are 66 words in the doc.

So, input / feature vector

$$x = \langle x_1, x_2, x_3, x_4, x_5, x_6 \rangle$$

$$= \langle 3, 2, 1, 3, 0, 4.19 \rangle$$

Probability of text/paragraph/doc containing positive emotion ,

$$P(+|x) = P(y=1|x) = \sigma(z)$$

$$= \sigma(w \cdot x + b)$$

↗ Class 1 = positive emotion class

$$= \sigma \left(\underbrace{\langle 2.5, -5.0, -1.2, 0.5, 2.0, 0.7 \rangle}_{\mathbf{w}} \cdot \underbrace{\langle 3, 2, 1, 3, 0, 4.19 \rangle}_{\mathbf{x}} + 0.1 \right)$$

dot multiplication

$$= \sigma \left((2.5 \times 3 - 5.0 \times 2 - 1.2 \times 1 + 0.5 \times 3 + 2.0 \times 0 + 0.7 \times 4.19) + 0.1 \right)$$

$$= \sigma(0.833)$$

$$= \frac{1}{1 + e^{-0.833}} \quad \left[\sigma(z) = \frac{1}{1 + e^{-z}} \right]$$

$$= 0.70$$

$\therefore$ probability of text containing negative emotion

$\rightarrow$ class 0 = negative emotion class

$$P(-|x) = P(y=0|x) = 1 - \sigma(w \cdot x + b)$$

$$= 1 - 0.70$$

$$= 0.30$$

$\therefore$ the text contains positive emotion.

In this example, the model / logistic regression classifier is given $\mathbf{x}$ and $\mathbf{w}$ and b . So, we did not have to find the value of $\mathbf{w}$ and b .

Processing multiple examples in matrix form!

Let us, assume,

we have a logistic regression model,

$$z = w_1 x_1 + w_2 x_2 + b \quad \text{and } y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

where weight vector $w = \langle w_1, w_2 \rangle = \begin{bmatrix} 0.4 \\ 0.6 \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$
and bias $b = -1$.

as we have two features x_1 and x_2 if we have one test case/example it will have both x_1 and x_2 value.

Let's assume test dataset $\rightarrow$

	x_1	x_2
$x^{(1)}$	2	1
$x^{(2)}$	3	4
$x^{(3)}$	5	2

$x_1^{(3)}$
 $\downarrow$
3rd test example's feature 1 value

$x_2^{(3)}$

$\rightarrow$ 3rd test example's feature 2 value.

$\therefore$ training input matrix $x = \begin{bmatrix} 2 & 1 \\ 3 & 4 \\ 5 & 2 \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ x_1^{(2)} & x_2^{(2)} \\ x_1^{(3)} & x_2^{(3)} \end{bmatrix}$

We know,
 Prediction $\hat{y} = \sigma(z)$
 $= \sigma(\underset{\substack{\text{weight vector} \\ \text{input matrix}}}{W \cdot X} + \underset{\text{bias}}{b})$

Let us calculate,

$$\begin{aligned}
 X \cdot W &= \begin{bmatrix} 2 & 1 \\ 3 & 4 \\ 5 & 2 \end{bmatrix}_{3 \times 2} \cdot \begin{bmatrix} 0.4 \\ 0.6 \end{bmatrix}_{2 \times 1} \\
 &= \begin{bmatrix} 2 \times 0.4 + 1 \times 0.6 \\ 3 \times 0.4 + 4 \times 0.6 \\ 5 \times 0.4 + 2 \times 0.6 \end{bmatrix} = \begin{bmatrix} 1.4 \\ 3.6 \\ 3.2 \end{bmatrix}_{3 \times 1}
 \end{aligned}$$

$$\therefore z = X \cdot W + b = \begin{bmatrix} 1.4 \\ 3.6 \\ 3.2 \end{bmatrix} + \underset{\text{bias vector}}{\begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix}} = \begin{bmatrix} 0.4 \\ 2.6 \\ 2.2 \end{bmatrix}$$

$$\begin{aligned}
 \therefore \hat{y} = \sigma(z) &= \sigma \left(\begin{bmatrix} 0.4 \\ 2.6 \\ 2.2 \end{bmatrix} \right) = \begin{bmatrix} \sigma(0.4) \\ \sigma(2.6) \\ \sigma(2.2) \end{bmatrix} \\
 &= \begin{bmatrix} 0.5987 \\ 0.9309 \\ 0.9002 \end{bmatrix} \quad \sigma(z) = \frac{1}{1 + e^{-z}}
 \end{aligned}$$

So, in general $\rightarrow$

if there are f features,

$$x_1, x_2, \dots, x_f$$

m test examples $\rightarrow$

1st test example $\leftarrow x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(m)}$

$$\text{input matrix } x = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_f^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_f^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \dots & x_f^{(m)} \end{bmatrix}$$

As there are f features, there will be f number of weights, as we know,

$$z = x_1 w_1 + x_2 w_2 + \dots + x_f w_f + b$$

$$\therefore w = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_f \end{bmatrix}$$

We need to predict class for m examples, so

$$\begin{aligned} P(y=1 | x^{(1)}) &= \text{Probability of class 1 for training example 1} \\ &= \sigma(w \cdot x^{(1)} + b) \end{aligned}$$

$$P(y^{(2)} = 1 | x^{(2)}) = \sigma(w \cdot x^{(2)} + b)$$

⋮

$$P(y^{(m)} = 1 | x^{(m)}) = \sigma(w \cdot x^{(m)} + b)$$

so, prediction y will be a vector as well.

$$\text{vector } \hat{y} = \sigma(x \cdot w + b)$$

$$= \sigma \left(\begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_f^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_f^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \dots & x_f^{(m)} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_f \end{bmatrix} + \begin{bmatrix} b \\ b \\ \vdots \\ b \end{bmatrix} \right)$$

$m \times f$ $f \times 1$ $m \times 1$

$$\Rightarrow y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} m \times 1$$

Finding the Logistic regression model:

To find the Logistic regression model,

$$\hat{y} = \sigma(z) = \sigma(w \cdot x + b) \\ = \sigma(w_1 x_1 + w_2 x_2 \dots w_n x_n + b)$$

We need to find the value of weight vector $\langle w_1, w_2, \dots, w_n \rangle$ and bias b . Then we can easily predict $\hat{y}$ vector.

In linear regression we have used Gradient Descent to optimize the weights ~~for~~. ~~we used~~

Here we will also use gradient descent to optimize the weight and bias, using ~~gradient descent~~.

We will start with a random value of weight w and b bias and iteratively update this parameters.

We know the parameter updates are done as below $\rightarrow$

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{d(\text{Loss function})}{dw_1}$$

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{d}{db} (\text{Loss function})$$

η
learning rate.

So, we need to derive a loss function for logistic regression.

Here in Logistic Regression we use a loss function called

Cross Entropy

Deriving the Loss Function (BCE):

The loss function will be used to update the parameter weights and bias. And we need to do this while training our model with dataset.

Let's say we have a training dataset with f features and m data points like below $\rightarrow$

f number of features

	x_1	x_2		x_f	target y
$x^{(1)}$	-	-		-	$y^{(1)}$
$x^{(2)}$	-	-		-	$y^{(2)}$
$\vdots$					$\vdots$
$x^{(m)}$					$y^{(m)}$

m number of datapoints / instances.

binary class
0/1

For each data point in the dataset,

Probability of being in a class for feature vector

$$x, P(y = \text{class} | x)$$

↓
actual $y^{(i)}$ from the dataset

for $y = 1$ class,

$$\text{Prediction } P(y=1|x) = \sigma(z) = \hat{y} \quad \text{--- (2)}$$

and for $y=0$ class,

$$\text{Prediction } P(y=0|x) = 1 - \sigma(z) \\ = 1 - \hat{y} \quad \text{--- (3)}$$

We can write a general equation for this two probabilities

$$P(y|x) = (\hat{y})^y (1 - \hat{y})^{1-y}$$

Actual y from dataset

Predicted $\hat{y} / \sigma(z)$

here if $y=1$, [if any data point has class labelled as 1]

$$\text{then prediction } P(y=1|x) = (\hat{y})^1 (1 - \hat{y})^{1-1} \\ = \hat{y}$$

[we get equation (2)]

and if $y=0$, [if any data point has class labelled as 0]

$$\text{then prediction } P(y=0|x) = (\hat{y})^0 (1-\hat{y})^{1-0}$$

$$= 1 \times 1 - \hat{y}$$

$$= 1 - \hat{y}$$

→ [we get equation ③]

So, this equation, $P(y|x) = (\hat{y})^y (1-\hat{y})^{1-y}$ generalizes probability for both class.

As we are finding probability of $y=1$ and $y=0$ class, we will not get exactly 1 and 0 as prediction probability from $P(y=1|x)$ and $P(y=0|x)$.

→ if $y=1$ class gets correctly classified as 1, the prediction probability $P(y=1|x) \in [0.5, 1]$

→ if $y=0$ class gets correctly classified as 0, the prediction probability $P(y=0|x) \in [0.5, 1]$

But we want to maximize these probabilities near to 1. So,

→ if any datapoint belongs to class 1, we want $P(y=1|x) = 1$ [perfectly 1]

→ if any data point belongs to class 0, we want

$$P(y=0|x) = 1 \text{ [perfectly 1]}$$

So, we want to maximize $P(y=1|x)$ and $P(y=0|x)$,

in general terms, we want to maximize,

$$P(y|x) = (\hat{y})^y (1-\hat{y})^{1-y}$$

In the dataset, there are m datapoints, we are training with m datapoints and trying to optimize w weights and b bias. So, for m datapoints the total prediction probability / Likelihood will be the joint probability of observing all training inputs.

So, total probability / likelihood for m datapoints,

$$P(y^{(1)}, y^{(2)}, \dots, y^{(m)} | x^{(1)}, x^{(2)}, \dots, x^{(m)})$$

Just like before we want to maximize this likelihood as well.

If we assume all m datapoints are independent, we can write.

$$P(y^{(1)}, y^{(2)}, \dots, y^{(m)} | x^{(1)}, x^{(2)}, \dots, x^{(m)})$$

$$= P(y^{(1)} | x^{(1)}) \times P(y^{(2)} | x^{(2)}) \times \dots \times P(y^{(m)} | x^{(m)})$$

$$= \prod_{i=1}^m P(y^{(i)} | x^{(i)})$$

$$= \prod_{i=1}^m (\hat{y}^{(i)})^{y^{(i)}} (1 - \hat{y}^{(i)})^{1-y^{(i)}}$$

So, we want to maximize this product to get the accurate w and b .

This product holds both $P(y=1|x) = (\hat{y}^{(i)})$ and $P(y=0|x) = 1 - \hat{y}^{(i)}$. It is because some data points are labelled as 1 and some as 0. We want to maximize both class probabilities to 1.

However, it is very difficult to calculate this multiplication, when m is very large. To simplify we take "log" [while calculation use 'ln'] of this product, which gives us summation in series in place of product. We find **log-likelihood** after taking log.

$$\begin{aligned} \log \text{likelihood} &= \sum_{i=1}^m \log [P(y^{(i)} | x^{(i)})] \\ &= \sum_{i=1}^m \log [(\hat{y}^{(i)})^{y^{(i)}} (1 - \hat{y}^{(i)})^{1-y^{(i)}}] \\ &= \sum_{i=1}^m \log(\hat{y}^{(i)})^{y^{(i)}} + \log(1 - \hat{y}^{(i)})^{1-y^{(i)}} \\ &= \sum_{i=1}^m y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \end{aligned}$$

Now want to maximize this log-likelihood.

Now if we take negative of this log-likelihood, we will want to minimize it instead of maximizing.

This minimizing function is our Cross

Entropy Loss function.

minus makes it minimization function.

$$\text{Loss function (CE)} = - \sum_{i=1}^m y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)})$$

When class $y^{(i)} = 1$, we want to minimize

$$\begin{aligned} & - \sum_{i=1}^m 1 \log \hat{y}^{(i)} + (1 - 1) \log (1 - \hat{y}^{(i)}) \\ & = - \sum_{i=1}^m \log \hat{y}^{(i)} \end{aligned}$$

When data points belongs to $y^{(i)} = 0$ class, we want to

minimize,

$$\begin{aligned} & - \sum_{i=1}^m 0 \times \log \hat{y}^{(i)} + (1 - 0) \log (1 - \hat{y}^{(i)}) \\ & = - \sum_{i=1}^m \log (1 - \hat{y}^{(i)}) \end{aligned}$$

Now let's check if this Loss function give us error while predicting correctly and incorrectly.

	feature vector x	Actual y	Predicted $\hat{y}$	
$x^{(1)}$	$< - >$	1	1	→ correct prediction
$x^{(2)}$	$< - >$	1	0	→ incorrect prediction
$x^{(3)}$	$< - >$	0	0	→ correct "
$x^{(4)}$	$< - >$	0	1	→ incorrect "

Let us assume, we have 4 data points in this dataset. and we have run our logistic regression model over this and found some predicted $\hat{y}$. Let us check if our loss function CE gives us Error correctly.

For datapoint $x^{(1)} \rightarrow y^{(1)} = 1$ and $\hat{y}^{(1)} = 1$,

$$\text{Loss (CE)} = -y^{(1)} \log \hat{y}^{(1)} - (1 - y^{(1)}) \log (1 - \hat{y}^{(1)})$$

$$= -1 \log 1 - (1 - 1) \log (1 - 1)$$

$$= -1 \log 1 = 0 \quad [\text{correct prediction Loss} = 0]$$

For datapoint $x^{(2)} \rightarrow y^{(2)} = 1$ and $\hat{y}^{(2)} = 0$

$$\text{Loss (CE)} = -y^{(2)} \log \hat{y}^{(2)} - (1 - y^{(2)}) \log (1 - \hat{y}^{(2)})$$

$$= -1 \log 0 - (1 - 1) \log (1 - 0)$$

$$= -\log 0 = \text{undefined} \quad [\text{in a range it's } -\infty]$$

$$[\text{incorrect prediction Loss} = \infty]$$

For datapoint $x^{(3)} \rightarrow y^{(3)} = 0, \hat{y}^{(3)} = 0$

$$\begin{aligned}\text{Loss(CE)} &= -y^{(3)} \log \hat{y}^{(3)} - (1-y^{(3)}) \log (1-\hat{y}^{(3)}) \\ &= -0 \log 0 - (1-0) \log (1-0) \\ &= -\log 1 = 0 \\ &\quad \text{[correct prediction Loss=0]}\end{aligned}$$

For datapoint $x^{(4)} \rightarrow y^{(4)} = 0, \hat{y}^{(4)} = 1$

$$\begin{aligned}\text{Loss(CE)} &= -y^{(4)} \log \hat{y}^{(4)} - (1-y^{(4)}) \log (1-\hat{y}^{(4)}) \\ &= -0 \log 1 - (1-0) \log (1-1) \\ &= -\log 0 \\ &= \text{undefined} \quad \text{[in a range it is -Infinity]} \\ &\quad \text{[incorrect prediction Loss=Infinity]}\end{aligned}$$

So, we can use this loss function for the gradient descent process to update the weights and bias parameters.

For updating, we need derivative of loss function as below,

$$\begin{aligned}&\frac{\partial \text{Loss function}}{\partial w_i} \rightarrow \text{every weight } w_i \\ &\frac{\partial \text{Loss function}}{\partial b} \rightarrow \text{bias.}\end{aligned}$$

Derivative of Loss function:

$$\text{Loss function } L_{CE} = - \sum_{i=1}^n y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log (1-\hat{y}^{(i)})$$

Let us consider it for one datapoint then,

$$L_{CE} = -y \log \hat{y} - (1-y) \log (1-\hat{y})$$

Let us assume, $z = w_1 x_1 + w_2 x_2 + b$ [For a problem]
of
derivative interms ~~of~~ $w_1 \rightarrow$

$$\frac{\partial L_{CE}}{\partial w_1} = \frac{\partial L_{CE}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial w_1}$$

$\hookrightarrow$ We do this because we don't have w_1 in L_{CE} .
So, we ~~derivate it~~ differentiate it ~~at~~ with respect to $\hat{y}$. So, we need to differentiate $\hat{y}$ now.

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

So, we can differentiate it with respect to z . then

Finally z has w_1 in it so, we differentiate it with respect to w_1 . This is the chain rule.

Let us do the differentiation part by part.

$$\begin{aligned}
 \frac{\partial L_{CE}}{\partial \hat{y}} &= \frac{\partial}{\partial \hat{y}} \left[-y \log \hat{y} - (1-y) \log (1-\hat{y}) \right] \\
 &= -y \times \frac{1}{\hat{y}} - \frac{1-y}{1-\hat{y}} \times \frac{\partial}{\partial \hat{y}} (1-\hat{y}) \\
 &= -\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}} \times (-1) \\
 &= -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \\
 &= \frac{-y + \cancel{y \times \hat{y}} + \hat{y} - \cancel{y \times \hat{y}}}{\hat{y}(1-\hat{y})} \\
 &= \frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \quad \text{--- (5)}
 \end{aligned}$$

$$\frac{\partial \hat{y}}{\partial z} = \frac{\partial}{\partial z} \sigma(z) = \sigma(z) (1 - \sigma(z)) \quad [\text{from (4)}]$$

$$\therefore \frac{\partial \hat{y}}{\partial z} = \hat{y} (1 - \hat{y}) \quad [\hat{y} = \sigma(z)] \quad \text{--- (6)}$$

$$\begin{aligned}
 \frac{\partial z}{\partial w_1} &= \frac{\partial}{\partial w_1} (w_1 x_1 + w_2 x_2 + b) \\
 &= x_1
 \end{aligned}$$

$$\begin{aligned}
 \therefore \frac{\partial L_{CE}}{\partial w_1} &= \frac{\partial L_{CE}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial w_1} \\
 &= \frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \times \cancel{\hat{y}(1-\hat{y})} \times x_1 \\
 &= (\hat{y} - y) \times x_1
 \end{aligned}$$

$$\therefore \frac{\partial L_{CE}}{\partial w_1} = (\hat{y} - y) x_1$$

similarly,

$$\frac{\partial L_{CE}}{\partial w_2} = \boxed{\frac{\partial L_{CE}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z}} \times \frac{\partial z}{\partial w_2} \rightarrow \text{need to find this}$$

we already know this part

$$\therefore \frac{\partial z}{\partial w_2} = \frac{\partial}{\partial w_2} (w_1 x_1 + w_2 x_2 + b_0)$$

$$= x_2$$

$$\therefore \frac{\partial L_{CE}}{\partial w_2} = \frac{\partial L_{CE}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial w_2}$$

$$= \frac{\hat{y} - y}{\cancel{y(1-\hat{y})}} \times \cancel{\hat{y}} \times \cancel{(1-\hat{y})} \times x_2$$

$$= (\hat{y} - y) x_2$$

→ we can see the derivative with respect to w_2 is same except as w_1 except the x term.

We can generalize derivative of L_{CE} with respect to weight w_i as →

$$\frac{\partial L_{CE}}{\partial w_i} = (\hat{y} - y) x_i$$

Now, derivative with respect to b :

$$\frac{\partial L_{EE}}{\partial b} = \left[\frac{\partial L_{EE}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \right] \times \frac{\partial z}{\partial b}$$

↳ this part is,

$$\frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \times \hat{y}(1-\hat{y})$$
$$= \boxed{\hat{y} - y}$$

$$\therefore \frac{\partial z}{\partial b} = \frac{\partial}{\partial b} (w_1 x_1 + w_2 x_2 + b) = 1$$

$$\therefore \frac{\partial L_{EE}}{\partial b} = \frac{\partial L_{EE}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial b}$$
$$= (\hat{y} - y) \times 1$$
$$= \hat{y} - y$$

Example-1:

x_1	x_2	y
3	2	1

Let's assume initial weight and bias are set to zero. Learning rate is 0.1. Update weights and bias ^[for one cycle] using gradient descent in one.

Solution:

for two feature x_1, x_2

$$z = w_1 x_1 + w_2 x_2 + b \quad \text{and} \quad \hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

So, we have to update two weights and b . Given,

$$\theta^0 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

we know, updates will be done as follows,

$$w_i (\text{new}) = w_i (\text{old}) + \eta \frac{\partial L_{CE}}{\partial w_i} \quad \left[\begin{array}{l} \text{Here old is } \theta^0 \\ \text{new will be } \theta^1 \end{array} \right]$$
$$b (\text{new}) = b (\text{old}) + \eta \frac{\partial L_{CE}}{\partial b}$$

$\therefore$ Gradient of Loss function with respect to w and b ,

$$\nabla_{w,b} L = \begin{bmatrix} \frac{\partial L_{CE}}{\partial w_1} \\ \frac{\partial L_{CE}}{\partial w_2} \\ \frac{\partial L_{CE}}{\partial b} \end{bmatrix} = \begin{bmatrix} (\hat{y} - y) x_1 \\ (\hat{y} - y) x_2 \\ (\hat{y} - y) \end{bmatrix} \quad \begin{array}{l} \hat{y} = \sigma(z) \\ = \sigma(w_1 x_1 + w_2 x_2 + b) \end{array}$$
$$= \begin{bmatrix} [\sigma(w_1 x_1 + w_2 x_2 + b) - y] x_1 \\ [\sigma(w_1 x_1 + w_2 x_2 + b) - y] x_2 \\ \sigma(w_1 x_1 + w_2 x_2 + b) - y \end{bmatrix}$$

$$= \begin{bmatrix} [\sigma(0) - 1] \times 3 \\ [\sigma(0) - 1] \times 2 \\ \sigma(0) - 1 \end{bmatrix} \quad \begin{bmatrix} z = w_1 x_1 + w_2 x_2 + b \\ = 0 \times 3 + 0 \times 2 + 0 \\ = 0 \end{bmatrix}$$

$$= \begin{bmatrix} -0.5 \times 3 \\ -0.5 \times 2 \\ -0.5 \end{bmatrix}$$

$$= \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

$$\sigma(0) = \frac{1}{1 + e^{-0}} = 0.5$$

Now we have the gradients, we can compute the new updated value of the weights and bias

$$\theta^1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \times \begin{bmatrix} \frac{\partial LCE}{\partial w_1} \\ \frac{\partial LCE}{\partial w_2} \\ \frac{\partial LCE}{\partial b} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - 0.1 \times \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.1 \\ 0.05 \end{bmatrix}$$

we can also calculate / update the weights individually instead of doing it in vector / matrix form.

Example - 2

Find the linear classifier for an OR gate using Logistic Regression.

The linear equation for a two input OR gate is $z = w \cdot x = w_1 x_1 + w_2 x_2 + b$. Let us assume,

$$w_1 = 0.5, w_2 = 0.75 \text{ and } b = -1.25, \quad \eta = 0.2$$

$$\text{initial } z = 0.5 x_1 + 0.75 x_2 - 1.25$$

Do two cycle of gradient descent and update w_1, w_2 and b .

Solution:

We know the truth table of OR gate $\rightarrow$

	x_1	x_2	y
① $\leftarrow$	0	0	0
② $\leftarrow$	0	1	1
③ $\leftarrow$	1	0	1
④ $\leftarrow$	1	1	1

$\downarrow \quad \downarrow \quad \rightarrow$ features actual target.

1st cycle $\rightarrow$

Data point ① $x_1 = 0, x_2 = 0$ and $y = 0$.

$$\therefore z = w_1 x_1 + w_2 x_2 + b$$

$$= 0.5 \times 0 + 0.75 \times 0 - 1.25$$

$$= -1.25$$

$$\text{and } \hat{y} = \sigma(z) = \sigma(-1.25) = 0.22 < 0.5$$

↓
Class is 0

So, our prediction/hypothesis $h_w(x) = 0$

∴ Actual class $y = 0$ and predicted class $h_w = 0$.

So we will not update ~~the~~ w_1, w_2 , and b .

Datapoint - 2 → $x_1 = 0, x_2 = 1$ and $y = 1$

$$z = 0.5x_1 + 0.75x_2 - 1.25$$

$$= 0.5 \times 0 + 0.75 \times 1 - 1.25 = -0.5$$

$$\text{and } \hat{y} = \sigma(z) = \sigma(-0.5) = 0.38 \rightarrow h_w = 0 \text{ as } \hat{y} < 0.5$$

but actual class $y = 1$. So we need to update w_1, w_2 and b .

$$\begin{aligned} \therefore w_1 &= w_1(\text{old}) - \eta \left[\frac{\partial L_{CE}}{\partial w_1} \right] = w_1(\text{old}) - \eta \left[(\hat{y} - y) x_1 \right] \\ &= 0.5 - 0.2 \times (0.38 - 1) \times 0 \\ &= 0.5 \end{aligned}$$

$$\begin{aligned} \therefore w_2 &= w_2(\text{old}) - \eta (\hat{y} - y) x_2 = 0.75 - 0.2 \times (0.38 - 1) \times 0 \\ &= 0.87 \end{aligned}$$

$$\begin{aligned} \therefore b &= b(\text{old}) - \eta \left[\frac{\partial L_{CE}}{\partial b} \right] = b(\text{old}) - \eta (\hat{y} - y) \\ &= -1.25 - 0.2 \times (0.38 - 1) \\ &= -1.13 \end{aligned}$$

$$\text{So, new } z = \overset{w_1(\text{new})}{0.5} x_1 + \overset{w_2(\text{new})}{0.87} x_2 + \overset{b(\text{new})}{-1.13}$$

Datapoint $\rightarrow$ ③ $x_1 = 1, x_2 = 0, y = 1$

$$z = 0.5 \times 1 + 0.87x_2 - 1.13 = -0.63$$

$$\hat{y} = \sigma(z) = \sigma(-0.63) = 0.35$$

$$\hookrightarrow hw(x) = 0 \text{ as } \hat{y} < 0.5$$

Actual class $y = 1$ and predicted class $hw(x) = 0$
So, ~~we~~ need to update weights and bias.

$$w_1(\text{new}) = w_1(\text{old}) - \eta \times (\hat{y} - y)x_1 = 0.5 - 0.2(0.35 - 1) \times 1 = 0.63$$

$$w_2(\text{new}) = w_2(\text{old}) - \eta \times (\hat{y} - y)x_2 = 0.87 - 0.2(0.35 - 1) \times 0 = 0.87$$

$$b(\text{new}) = b(\text{old}) - \eta \times (\hat{y} - y) = -1.13 - 0.2(0.35 - 1) = -1.0$$

$$\therefore \text{So, } z = 0.63x_1 + 0.87x_2 - 1.0$$

Datapoint ④ $\rightarrow x_1 = 1, x_2 = 1$ and $y = 1$

$$z = 0.63 \times 1 + 0.87 \times 1 - 1.0 = 0.5$$

$$\hat{y} = \sigma(z) = \sigma(0.5) = 0.62 > 0.5$$

$$\therefore hw(x) = 1$$

Actual class $y = 1$ and predicted class $hw(x) = 1$
are same.

So, no need to update the weights and bias.

After first cycle

$$\begin{aligned} \tilde{x} &= 0.87 \\ w_1 &= 0.63 \\ w_2 &= 0.87 \\ b &= -1.0 \end{aligned}$$

$$\text{And } z = 0.63x_1 + 0.87x_2 - 1.0$$

Similarly we can do the ~~same~~ second cycle for datapoint ① $\rightarrow$ ④ Again.